TRABAJO DE INVESTIGACIÓN · TEMAS 14 Y 25



Docker Compose y Testcontainers

Reproducir y probar un sistema distribuido con contenedores

INTEGRANTES

Villalba, Luca
Matta, Heber
Carrasco, Agustin
Saavedra, Diego
Stegmayer, Mauro
Arechavala, Santino
Gossen, Ian
Ozuna, Lautaro
Ghione, Lucas

Grupo 10 · Módulo 8: Notificaciones, Documentos y Soporte

Plataforma Distribuida de Movilidad Urbana

Desarrollo de Software · UTN · 2026

Noviembre de 2026

Contenido

1. Introducción, alcance y fundamentos de Docker
2. Docker Compose para un sistema distribuido (Tema 14)
3. Testing de integración con Testcontainers (Tema 25)
4. Relación con el Trabajo Práctico Integrador
5. Criterios de utilización
6. Implementación práctica integrada y evidencias
7. Decisiones de diseño y hallazgos
8. Cómo reproducir la demostración
9. Conclusiones
10. Bibliografía

Resumen

El Grupo 10 investigó dos temas complementarios. **Docker Compose** describe en un archivo todo el sistema del Módulo 8 (dos APIs, PostgreSQL, Redis, RabbitMQ, Mailpit y el frontend) y lo levanta con un solo comando. **Testcontainers** usa la misma tecnología de contenedores para crear, desde el código de las pruebas, una base PostgreSQL real y descartable. Ambos comparten la misma imagen (`postgres:16-alpine`) y el mismo esquema de datos: Compose sirve para *operar y demostrar* el sistema completo, y Testcontainers para *verificar automáticamente* una parte de él.

1. Introducción y alcance

La cátedra asignó al Grupo 10 los temas **14 (Docker Compose para un sistema distribuido)** y **25 (Testing de integración con Testcontainers)**. Las preguntas guía son:

- **Tema 14:** ¿cómo reproducir y operar localmente un sistema con varios servicios, dependencias y configuraciones usando Docker Compose?
- **Tema 25:** ¿cómo ejecutar pruebas de integración reproducibles utilizando dependencias reales y aisladas?

Los dos temas atacan el mismo problema desde lugares distintos. Un sistema distribuido depende de piezas externas (bases de datos, colas, cachés) y cada integrante las tendría que instalar y configurar igual que los demás. Los contenedores resuelven eso empaquetando cada pieza con todo lo que necesita. Compose aplica la idea al **entorno de ejecución**; Testcontainers, al **entorno de pruebas**.

El escenario elegido es el propio Módulo 8 (M8) de la plataforma de movilidad urbana: notificaciones, comprobantes en PDF, códigos QR y tickets de soporte. Todo lo que se describe en las secciones 4 a 8 existe en el repositorio del grupo (`FRRe-DS/2026-10-TPI`).

Como los dos temas se apoyan en Docker, los apartados 1.1 a 1.7 repasan antes qué es, qué problema resuelve y qué conceptos usa el resto del trabajo.

1.1 El problema: «en mi máquina funciona»

Un programa no corre solo. Necesita una versión determinada de su entorno de ejecución (en el M8, Node.js 20), bibliotecas, variables de configuración y servicios externos. Cuando cada desarrollador prepara eso a mano en su computadora, los entornos terminan siendo distintos, y aparece la frase clásica: *«en mi máquina funciona»*.

En el M8 el problema es concreto: el backend necesita PostgreSQL 16, Redis 7, RabbitMQ y un servidor de correo de prueba. Sin contenedores, cada uno de los nueve integrantes tendría que instalar las cuatro piezas en su sistema operativo. Eso trae varios problemas:

- **Versiones distintas:** un integrante con PostgreSQL 14 y otro con 16 pueden obtener resultados diferentes con el mismo código.
- **Configuración no escrita:** puertos, usuarios y contraseñas quedan en la memoria de quien instaló, no en el repositorio.
- **Conflictos:** dos proyectos que necesitan versiones distintas de la misma base no pueden convivir fácilmente en la misma PC.
- **Incorporación lenta y entornos que divergen:** preparar una PC nueva lleva horas, y lo que se prueba en desarrollo no es igual a lo que corre en el CI o en producción [9].
- **Restos difíciles de limpiar:** desinstalar un motor de base de datos deja servicios, archivos y configuraciones en el sistema.

1.2 Qué es un contenedor

Un **contenedor** es una unidad estándar de software que empaqueta una aplicación junto con todo lo que necesita para ejecutarse: el código, el entorno de ejecución, las bibliotecas y la configuración [13]. Para el sistema operativo es un **proceso aislado**: tiene su propio sistema de archivos, su propia red y su propia lista de procesos, aunque corre sobre el mismo núcleo (*kernel*) que el resto de la máquina.

El aislamiento lo da el propio kernel de Linux con dos mecanismos: los **namespaces**, que limitan lo que el proceso *ve* (procesos, red, archivos, usuarios), y los **cgroups**, que limitan lo que puede *consumir* (CPU, memoria, disco). La analogía habitual es el contenedor de carga: tiene un formato estándar, así que el barco o el camión

lo transporta sin importar qué lleva adentro. Del mismo modo, un contenedor de software corre igual en la PC de cualquier integrante, en el CI o en un servidor.

Un contenedor es **autocontenido** (no depende de nada instalado en el host), **aislado** (no interfiere con otros), **independiente** (se crea y se borra sin afectar al resto) y **portable** (corre en cualquier máquina con un motor de contenedores).

1.3 Contenedores y máquinas virtuales

Una máquina virtual también aísla aplicaciones, pero lo hace en otro nivel: virtualiza el hardware y ejecuta un sistema operativo completo por cada máquina. Un contenedor, en cambio, virtualiza el sistema operativo: comparte el kernel del host y solo lleva lo que la aplicación necesita.



Figura 1. Una máquina virtual incluye un sistema operativo invitado completo; los contenedores comparten el kernel del host y solo empaquetan la aplicación y sus bibliotecas.

Aspecto	Máquina virtual	Contenedor
Qué virtualiza	El hardware (mediante un hipervisor).	El sistema operativo (kernel compartido).
Sistema operativo	Uno completo por cada máquina.	Ninguno propio: usa el del host.
Tamaño típico	Varios gigabytes.	Decenas o cientos de megabytes.
Arranque	Minutos (debe iniciar un sistema operativo).	Segundos (inicia un proceso).
Aislamiento	Más fuerte: cada máquina tiene su propio kernel.	Menor: un fallo del kernel del host afecta a todos.
Densidad	Pocas por servidor.	Decenas o cientos por servidor.

Las dos tecnologías no se excluyen. En Windows, que es lo que usa el grupo, los contenedores son de Linux y necesitan un kernel Linux: Docker Desktop lo obtiene de una máquina virtual liviana de WSL2, y dentro de ella corren todos los contenedores del M8.

1.4 Qué es Docker

Docker es una plataforma abierta para **construir, distribuir y ejecutar** aplicaciones en contenedores [14]. Los contenedores ya existían en Linux, pero eran difíciles de usar. Docker, presentado en 2013, los volvió prácticos: definió un formato de imagen fácil de construir, una línea de comandos simple y un registro público para compartir imágenes [16]. En 2015 sus formatos dieron origen a la **Open Container Initiative (OCI)**, que estandariza imágenes y entornos de ejecución [15]. Por eso una imagen construida con Docker también corre en Podman, containerd o Kubernetes.

Docker usa una arquitectura cliente-servidor:

- **Cliente** (`docker`): la línea de comandos donde se escriben las órdenes, como `docker run` o `docker compose up`.
- **Daemon** (`dockerd`): el servicio que recibe esas órdenes por una API y hace el trabajo: construye imágenes y crea, ejecuta y detiene contenedores, redes y volúmenes.
- **Registro**: el lugar donde se guardan y descargan las imágenes. El público es Docker Hub, de donde salen `postgres`, `redis` y `rabbitmq`.
- **Docker Desktop**: la aplicación para Windows y macOS que reúne el daemon, el cliente, Compose y la máquina virtual Linux.

1.5 Conceptos básicos

Las secciones siguientes usan estos términos. Cada uno se acompaña con su ejemplo en el M8:

Concepto	Qué es	En el M8
Imagen	Plantilla de solo lectura, formada por capas, con todo lo necesario para crear un contenedor.	`postgres:16-alpine`, `redis:7-alpine` y la imagen del backend.
Contenedor	Instancia en ejecución de una imagen. Suma una capa escribible que se pierde al borrarlo.	`g10_m8_backend`, `g10_m8_postgres`, etc.
Dockerfile	Archivo de texto con los pasos para construir una imagen.	`Dockerfile` en la raíz: Node 20 Alpine, dos etapas y usuario no privilegiado.
Etiqueta (tag)	Nombre de una versión de la imagen.	`16-alpine`: PostgreSQL 16 sobre Alpine Linux.
Registro	Servidor que almacena y distribuye imágenes.	Docker Hub.
Volumen	Almacenamiento administrado por Docker que sobrevive al contenedor.	`postgres_data`, `redis_data`.
Red	Conecta contenedores y les permite encontrarse por nombre.	`m8_internal_net`, `microservices_net`.

El ciclo básico de Docker es **Dockerfile → imagen → contenedor**. Con un solo servicio alcanza con unos pocos comandos:

```
docker build -t g10-m8-backend .                  # Dockerfile -> imagen
docker pull postgres:16-alpine                    # descarga del registro
docker run -d --name db -p 5432:5432 \
  -e POSTGRES_PASSWORD=secreto postgres:16-alpine  # imagen -> contenedor
docker ps                                         # contenedores en ejecución
docker logs db                                    # salida del proceso
docker stop db && docker rm db                    # detener y borrar
```

Una consecuencia importante: lo que el contenedor escribe se guarda en su capa escribible y desaparece al borrarlo. Por eso las bases de datos guardan sus archivos en un **volumen**, fuera del contenedor (sección 2.2).

1.6 Para qué se usa

- **Entornos de desarrollo reproducibles:** todos los integrantes ejecutan las mismas versiones sin instalarlas en su sistema.
- **Integración continua:** el pipeline construye la imagen y prueba exactamente lo que luego se despliega. El CI del M8 ya valida la construcción de la imagen del backend.

- **Microservicios:** cada servicio viaja con su propio entorno y sus versiones, sin conflictos con los demás. Es el caso de la plataforma de movilidad, dividida en módulos.
- **Despliegue:** la misma imagen pasa de desarrollo a pruebas y a producción, lo que acerca los entornos [9].
- **Pruebas con dependencias reales:** se levanta una base de datos real para una prueba y se la tira al terminar. Es la idea de Testcontainers (sección 3).
- **Probar software sin instalarlo:** otra versión de PostgreSQL, por ejemplo, se prueba con un `docker run` y se elimina sin dejar rastros.

1.7 Límites de Docker por sí solo

- **Kernel compartido:** el aislamiento es menor que el de una máquina virtual, así que no conviene ejecutar imágenes de origen desconocido.
- **Contenedores Linux en Windows y macOS:** necesitan una máquina virtual, lo que suma consumo de memoria y algo de complejidad.
- **Datos efímeros:** sin volúmenes, los datos se pierden al borrar el contenedor.
- **Un comando por contenedor:** `docker run` maneja una pieza a la vez. Con siete servicios hay que recordar redes, puertos, variables y orden de arranque.

De Docker a Compose y Testcontainers

Docker resuelve el problema para **una** pieza. El M8 tiene siete, y además necesita pruebas contra una base real. Los dos temas de este trabajo son las respuestas a esos dos pasos: **Docker Compose** describe y levanta el sistema completo con un archivo (sección 2), y **Testcontainers** crea y destruye contenedores desde el código de las pruebas (sección 3).

2. Docker Compose para un sistema distribuido (Tema 14)

2.1 Qué es y qué problema resuelve

Docker Compose es una herramienta para **definir y ejecutar aplicaciones formadas por varios contenedores**. La aplicación se describe en un archivo YAML (`compose.yaml` o `docker-compose.yml`) que sigue la *Compose Specification*, y se administra con el comando `docker compose`.

Sin Compose, levantar el M8 exige ejecutar a mano un `docker run` por cada pieza, recordando puertos, variables, redes y el orden de arranque. Compose reemplaza esa secuencia por una descripción **declarativa y versionada**: el archivo vive en el repositorio y cualquier integrante obtiene el mismo entorno con `docker compose up`.

2.2 Conceptos principales

Concepto	Para qué sirve	En nuestro archivo
Servicio	Un contenedor (o varias réplicas) creado a partir de una imagen o de un Dockerfile.	7 servicios: `backend`, `api-viajes`, `frontend`, `postgres`, `redis`, `rabbitmq`, `mailpit`.
Red	Conecta contenedores entre sí. Dentro de una red, cada servicio se resuelve por su nombre mediante el DNS interno de Docker.	`m8_internal_net` y `microservices_net`.
Volumen	Guarda datos fuera del ciclo de vida del contenedor, para que sobrevivan a un reinicio o a un down.	`postgres_data`, `redis_data`, `rabbitmq_data`.
Healthcheck	Comando que Docker ejecuta periódicamente para saber si el servicio realmente responde.	`pg_isready`, `redis-cli ping`, `wget /api/v1/health`.
depends_on	Ordena el arranque. Con `condition: service_healthy` espera a que la dependencia esté sana.	El backend espera a PostgreSQL, Redis y RabbitMQ; las otras APIs esperan al backend.
Variables	Configuración por entorno, con `environment` y `env_file`.	El backend recibe los nombres de servicio en lugar de `localhost`.
restart	Política de reinicio cuando el proceso del contenedor termina.	`unless-stopped` en todos los servicios.

2.3 Beneficios y limitaciones

Beneficios	Limitaciones
Reproducibilidad: el entorno está escrito y versionado; desaparece el «en mi máquina funciona».	**Un solo host:** Compose no reparte contenedores entre varias máquinas ni escala automáticamente.
Un comando para levantar, detener o reconstruir todo el sistema.	**No es un orquestador:** no ofrece despliegues progresivos ni recuperación entre nodos como Kubernetes.
Aislamiento: redes y volúmenes propios por proyecto.	`depends_on` **solo ordena el arranque:** si una dependencia cae después, la aplicación debe saber reintentar.
Paridad entre desarrollo, CI y demostración (mismas imágenes y versiones).	**Un contenedor *unhealthy* no se reinicia solo:** la política `restart` actúa cuando el proceso termina, no cuando falla el healthcheck.
Documentación viva: el archivo muestra qué piezas tiene el sistema y cómo se conectan.	**Consumo de recursos** y secretos en texto plano si no se toman recaudos.

3. Testing de integración con Testcontainers (Tema 25)

3.1 Qué es y qué problema resuelve

Una **prueba de integración** verifica que varias piezas funcionen juntas: por ejemplo, que el código, el ORM, el esquema SQL y el motor de base de datos se entiendan. Las pruebas unitarias del M8 reemplazan esas piezas por *dobles de prueba* (mocks): son rápidas, pero no pueden demostrar que una restricción `UNIQUE`, un bloqueo o una migración se comporten bien en PostgreSQL.

Las alternativas clásicas tienen problemas. Una **base compartida** de pruebas se ensucia con datos de otros y vuelve las pruebas inestables. Una **base en memoria** (H2, SQLite) es otro motor, con otro dialecto SQL. **Testcontainers** es una biblioteca de código abierto que levanta dependencias **reales y descartables** en contenedores, directamente desde el código de la prueba, y las elimina al terminar.

3.2 Cómo funciona



Puerto del host asignado al azar · credenciales aleatorias por corrida · Ryuk elimina contenedores huérfanos

Figura 2. Ciclo de vida de la suite de integración del M8.

- **Contenedores por código:** la prueba pide una imagen (por ejemplo `PostgreSqlContainer('postgres:16-alpine')`) y la biblioteca habla con Docker para crearla.
- **Puertos dinámicos:** Docker asigna un puerto libre del host, así las pruebas no chocan con otros servicios ni entre sí.
- **Estrategias de espera:** la biblioteca espera a que el servicio esté listo antes de devolver el control.
- **Limpieza automática:** al terminar se detiene y elimina el contenedor; un contenedor auxiliar (*Ryuk*) borra lo que quede huérfano si el proceso muere.
- **Módulos listos** para PostgreSQL, Redis, RabbitMQ y Kafka, entre otros, en Java, Node.js, Go, Python y .NET.

3.3 Beneficios y limitaciones

Beneficios	Limitaciones
Fidelidad: se prueba contra el mismo motor y versión que usa el sistema.	**Requiere Docker** (o un runtime compatible) en cada PC y en el CI.
Aislamiento: cada corrida tiene su propia base, sin datos de otros.	**Más lento** que una prueba unitaria: arranque del contenedor y descarga inicial de la imagen.
Reproducibilidad: no hay que instalar ni configurar PostgreSQL.	**Consume CPU, memoria y disco**; no conviene abusar de la cantidad de contenedores.
Sin residuos: el entorno nace y muere con la prueba.	**Cobertura acotada:** solo valida lo que realmente se levanta.

4. Relación con el Trabajo Práctico Integrador

La plataforma de movilidad urbana se divide en módulos desarrollados por grupos distintos. El M8 recibe hitos de viajes, reservas y pagos (M5, M6, M7 y M9), notifica a los usuarios, genera comprobantes y administra tickets. Para funcionar necesita cuatro dependencias de infraestructura, y para integrarse necesita hablar con otros módulos.

4.1 Docker Compose en el M8

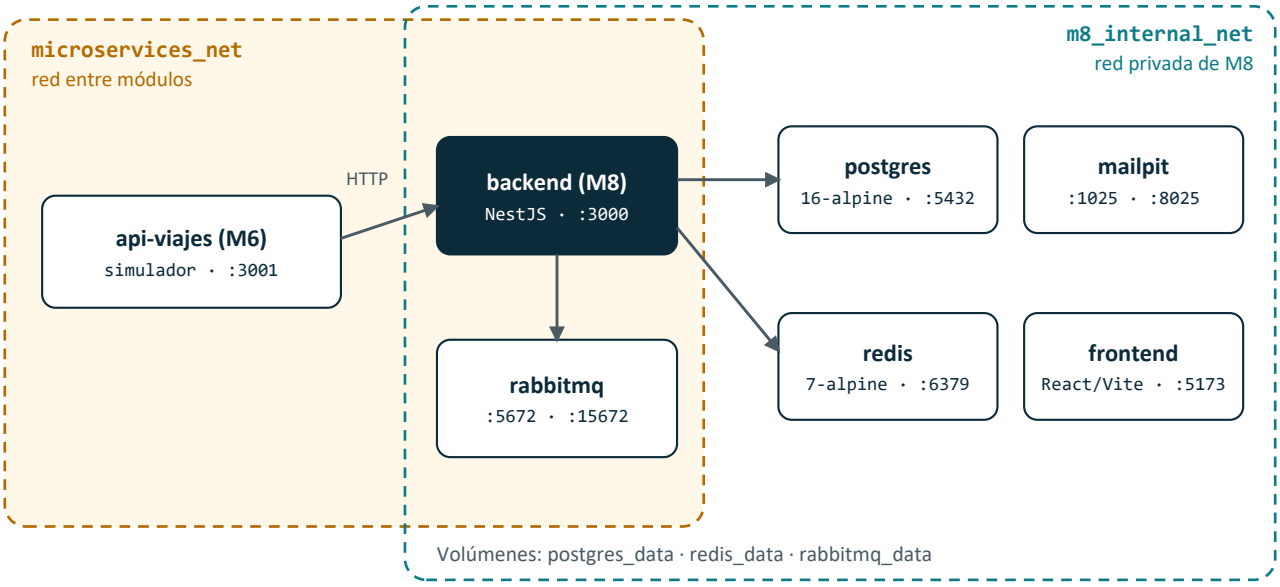


Figura 3. Servicios del `docker-compose.yml` *y las dos redes.* `backend` *y* `rabbitmq` *pertenecen a ambas.*

Servicio	Rol en la plataforma	Requerimiento
backend	API NestJS del M8: notificaciones, documentos, soporte y simulador.	RF-8.1 a RF-8.8
api-viajes	Segunda API: simulador del Módulo 6 que informa viajes finalizados al M8 por HTTP.	Integración entre módulos
postgres	Persistencia de comprobantes, tickets y registro de notificaciones.	RF-8.4, RF-8.6, RF-8.7
redis	Códigos QR de un solo uso con vencimiento (TTL).	RF-8.3
rabbitmq	Recepción asíncrona de eventos de otros módulos.	RF-8.8
mailpit	Servidor SMTP de prueba: captura los correos sin enviarlos.	RF-8.1, RF-8.5
frontend	Panel web de operadores (React + Vite).	Operación del M8

Las redes reflejan los límites del módulo. `m8_internal_net` es privada: allí viven la base y el caché, que ningún otro módulo debería tocar (cada servicio es dueño de sus datos). `microservices_net` representa el canal entre módulos: allí están la API de Viajes, el backend y RabbitMQ, que es por donde otros módulos envían eventos.

4.2 Testcontainers en el M8

Se eligieron los dos flujos cuya lógica depende de la base real:

Flujo	Por qué necesita PostgreSQL real	Casos
Comprobantes (RF-8.4)	La idempotencia por viaje se resuelve con `pg_advisory_xact_lock`. Con mocks solo se podía *afirmar* que no hay duplicados; con la base real se puede *demostrar*, enviando 8 solicitudes simultáneas.	7
Tickets de soporte (RF-8.7)	Alta, consulta y transiciones de estado (`OPEN → IN_PROGRESS → RESOLVED → CLOSED`) persistidas y validadas contra el esquema creado por las migraciones.	5

La suite levanta la aplicación NestJS completa en memoria y la ejercita por **HTTP real** (supertest). El esquema se crea con las **migraciones existentes** (`prisma migrate deploy`), de modo que también se prueba que esas migraciones se aplican sobre PostgreSQL 16.

5. Criterios de utilización

5.1 Docker Compose

Conviene usarlo cuando…	No se recomienda cuando…
El sistema tiene varias piezas que deben levantarse juntas en una sola máquina (desarrollo, demostraciones, CI).	Se necesita alta disponibilidad, escalado automático o varios nodos: ahí corresponde un orquestador (Kubernetes, Nomad, ECS).
Se quiere que todo el equipo use las mismas versiones sin instalar nada más que Docker.	La aplicación es un único proceso sin dependencias: Compose agrega complejidad sin beneficio.
Hace falta un entorno de integración entre módulos antes de desplegar.	Las dependencias ya son servicios administrados en la nube y no se necesita una copia local.

Factores: la complejidad de aprendizaje es baja y el costo es el de una PC con Docker. El rendimiento en Windows y macOS es algo menor porque Docker corre sobre una máquina virtual. La mantenibilidad mejora porque la infraestructura queda documentada en un archivo, aunque hay que mantenerlo alineado con el entorno productivo.

5.2 Testcontainers

Conviene usarlo cuando…	No se recomienda cuando…
La lógica depende de comportamiento real de la dependencia: bloqueos, restricciones, tipos, transacciones, migraciones.	Se prueba lógica pura de negocio: una prueba unitaria es más rápida y más clara.
Se necesitan pruebas repetibles sin una base compartida ni instalaciones locales.	El CI no puede ejecutar Docker o el tiempo de arranque no es aceptable para el ciclo de trabajo.
Se quiere validar las migraciones antes de aplicarlas en un entorno real.	Se pretende probar el sistema completo de punta a punta: para eso sirve un entorno con Compose.

Factores: cada corrida paga segundos de arranque y memoria, por lo que conviene pocas pruebas de integración bien elegidas sobre una base amplia de pruebas unitarias (pirámide de pruebas). En el M8 se comparte un contenedor por corrida y se vacían las tablas entre casos para no pagar ese costo en cada prueba.

5.3 Comparación directa

	Docker Compose (Tema 14)	Testcontainers (Tema 25)
Propósito	Levantar todo el sistema para desarrollo y demostración.	Levantar solo lo necesario para una prueba automatizada.
Quién lo controla	Una persona, con `docker compose up/down`.	El código de la prueba (Jest).
Ciclo de vida	Manual y duradero.	Efímero: nace y muere con la prueba.
Puertos	Fijos (5432, 6379, 3000…).	Dinámicos, sin colisiones.
Datos	Persisten en volúmenes.	Se descartan.
Qué demuestra	Que el sistema arranca, se comunica y resiste fallas.	Que una funcionalidad es correcta contra la dependencia real.

5.4 Alternativas consideradas

- **Instalar las dependencias en cada PC:** versiones distintas y configuración manual. Descartado.
- **Usar solo la base en la nube (Supabase):** sirve para trabajar sin Docker, pero es compartida, depende de la conexión y no es apta para pruebas destructivas. El grupo la mantiene como opción de desarrollo, no de pruebas.
- **Mocks o base en memoria:** se mantienen para las pruebas unitarias; no reemplazan a la base real.
- **Kubernetes local (minikube, kind):** excede lo que el TPI necesita.
- **Testcontainers con el archivo Compose** (`DockerComposeEnvironment`): permitiría levantar el entorno completo desde una prueba. Queda como mejora futura.

6. Implementación práctica integrada y evidencias

El escenario integra ambos temas alrededor de un mismo caso: **el comprobante de un viaje finalizado**. Con Compose se levanta el sistema completo y se muestra el recorrido real entre contenedores; con Testcontainers se verifica automáticamente la parte crítica de ese recorrido (la persistencia del comprobante) contra una base PostgreSQL descartable de la misma versión.

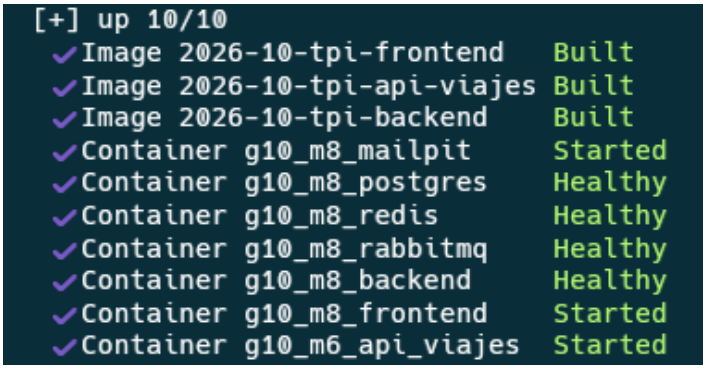
Cómo interactúan los dos temas

1) Misma imagen: `postgres:16-alpine` en Compose y en las pruebas. 2) Mismo esquema de Prisma: el contenedor del backend lo sincroniza al arrancar con `prisma db push` y las pruebas lo crean con las migraciones (`prisma migrate deploy`). 3) Mismo código de aplicación (`AppModule`). 4) Conviven sin conflicto: Compose usa el puerto 5432 y Testcontainers uno aleatorio.

6.1 Levantar el sistema con un comando

```
docker compose up -d --build
```

`-d` deja la consola libre y `--build` reconstruye las imágenes con el código actual.



Evidencia 1. Tres imágenes construidas y siete contenedores iniciados; las dependencias con healthcheck figuran como Healthy.

```
docker compose ps
NAME                IMAGE                          COMMAND                  SERVICE       CREATED              STATUS
g10_m6_api_viajes   2026-10-tpi-api-viajes         "docker-entrypoint.s…"   api-viajes    About a minute ago   Up 35 seconds (healthy)
g10_m8_backend      2026-10-tpi-backend            "docker-entrypoint.s…"   backend       About a minute ago   Up 41 seconds (healthy)
g10_m8_frontend     2026-10-tpi-frontend           "docker-entrypoint.s…"   frontend      About a minute ago   Up 35 seconds
g10_m8_mailpit      axllent/mailpit:latest         "/mailpit"               mailpit       About a minute ago   Up About a minute (healthy)
g10_m8_postgres     postgres:16-alpine             "docker-entrypoint.s…"   postgres      About a minute ago   Up About a minute (healthy)
g10_m8_rabbitmq     rabbitmq:3-management-alpine   "docker-entrypoint.s…"   rabbitmq      About a minute ago   Up About a minute (healthy)
g10_m8_redis        redis:7-alpine                 "docker-entrypoint.s…"   redis         About a minute ago   Up About a minute (healthy)
```

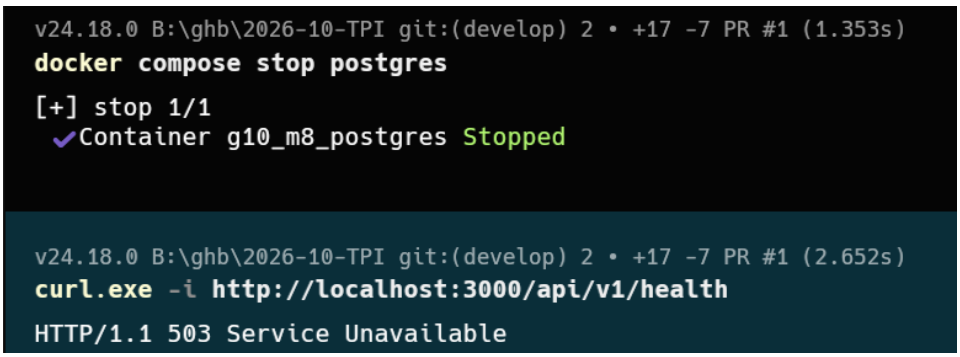
Evidencia 2. `docker compose ps`*: todos los servicios en ejecución.*

6.2 Healthchecks y falla de una dependencia

El endpoint `/api/v1/health` del backend consulta PostgreSQL y Redis. Docker lo usa como healthcheck del contenedor.

```
// Si Postgres no responde, devolvemos 503 para que Docker marque el contenedor como unhealthy
if (estadoPostgres === 'caido') {
  throw new ServiceUnavailableException(
    'La base de datos PostgreSQL no está disponible',
  );
}      You, 7 minutes ago • Uncommitted changes
```

Evidencia 3. `health.controller.ts`*: si PostgreSQL no responde, el backend devuelve 503.*



Evidencia 4. Con PostgreSQL detenido, el health responde `503 Service Unavailable`.

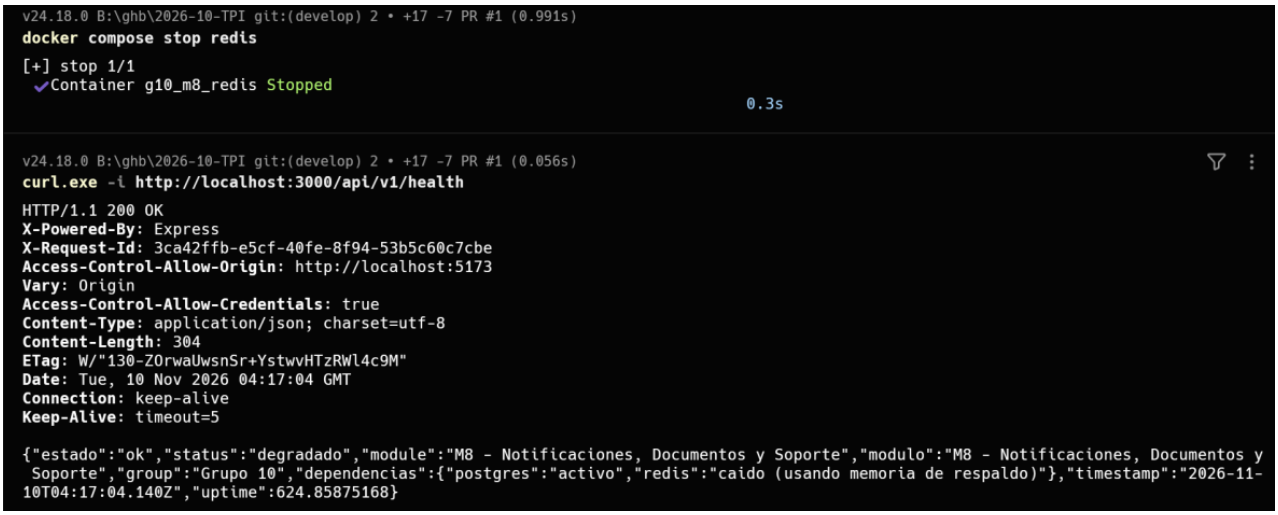


Evidencia 5. Docker marca el backend como unhealthy mientras la base está caída.



Evidencia 6. Al volver PostgreSQL, el backend recupera el estado healthy sin intervención.

Con Redis el comportamiento es distinto por decisión de diseño: el backend conmuta a una memoria local de respaldo y sigue atendiendo.



Evidencia 7. Con Redis detenido el health responde 200 con `status: degradado` *y el detalle de la dependencia caída.*

```
v24.18.0 B:\ghb\2026-10-TPI git:(develop) 2 • +17 -7 PR #1 (1.192s)
docker compose restart redis

[+] restart 0/1
 - Container g10_m8_redis Restarting

v24.18.0 B:\ghb\2026-10-TPI git:(develop) 2 • +17 -7 PR #1 (0.849s)
docker compose logs backend --tail 10

g10_m8_backend  | [Nest] 1  - 11/10/2026, 4:20:20 AM     LOG [PrismaService] Conexión establecida con la base de datos
g10_m8_backend  | [Nest] 1  - 11/10/2026, 4:20:20 AM     LOG [RedisService] Conexión con Redis restaurada (ready)
g10_m8_backend  | [Nest] 1  - 11/10/2026, 4:20:20 AM     LOG [RedisService] Conectado a Redis en redis:6379
g10_m8_backend  | [Nest] 1  - 11/10/2026, 4:20:20 AM     LOG [PrismaService] Conexión establecida con la base de datos
g10_m8_backend  | [Nest] 1  - 11/10/2026, 4:20:20 AM     LOG [PrismaService] Conexión establecida con la base de datos
g10_m8_backend  | [Nest] 1  - 11/10/2026, 4:20:20 AM     LOG [NestApplication] Nest application successfully started +1ms
g10_m8_backend  | [Nest] 1  - 11/10/2026, 4:20:20 AM     LOG [Bootstrap] Servidor M8 iniciado en puerto 3000 (api/v1)
g10_m8_backend  | [Nest] 1  - 11/10/2026, 4:20:20 AM     LOG [Bootstrap] Documentación Swagger en http://localhost:3000/api/docs
g10_m8_backend  | [Nest] 1  - 11/10/2026, 4:20:38 AM     LOG [RedisService] Conexión con Redis restaurada (ready)
g10_m8_backend  | [Nest] 1  - 11/10/2026, 4:21:35 AM     LOG [RedisService] Conexión con Redis restaurada (ready)
```

Evidencia 8. Reinicio de Redis: el backend registra «Conexión con Redis restaurada».

```
v24.18.0 B:\ghb\2026-10-TPI git:(develop) 2 • +17 -7 PR #1 (0.987s)
docker compose stop redis

[+] stop 1/1
 ✔Container g10_m8_redis Stopped

v24.18.0 B:\ghb\2026-10-TPI git:(develop) 2 • +17 -7 PR #1 (0.841s)
docker compose logs backend --tail 10

g10_m8_backend  | [Nest] 1  - 11/10/2026, 4:20:20 AM     LOG [RedisService] Conexión con Redis restaurada (ready)
g10_m8_backend  | [Nest] 1  - 11/10/2026, 4:20:20 AM     LOG [RedisService] Conectado a Redis en redis:6379
g10_m8_backend  | [Nest] 1  - 11/10/2026, 4:20:20 AM     LOG [PrismaService] Conexión establecida con la base de datos
g10_m8_backend  | [Nest] 1  - 11/10/2026, 4:20:20 AM     LOG [PrismaService] Conexión establecida con la base de datos
g10_m8_backend  | [Nest] 1  - 11/10/2026, 4:20:20 AM     LOG [NestApplication] Nest application successfully started +1ms
g10_m8_backend  | [Nest] 1  - 11/10/2026, 4:20:20 AM     LOG [Bootstrap] Servidor M8 iniciado en puerto 3000 (api/v1)
g10_m8_backend  | [Nest] 1  - 11/10/2026, 4:20:20 AM     LOG [Bootstrap] Documentación Swagger en http://localhost:3000/api/docs
g10_m8_backend  | [Nest] 1  - 11/10/2026, 4:20:38 AM     LOG [RedisService] Conexión con Redis restaurada (ready)
g10_m8_backend  | [Nest] 1  - 11/10/2026, 4:21:35 AM     LOG [RedisService] Conexión con Redis restaurada (ready)
g10_m8_backend  | [Nest] 1  - 11/10/2026, 4:22:32 AM    WARN [RedisService] Evento de error en Redis (getaddrinfo ENOTFOUND redis). Conmutando a memoria local.
```

Evidencia 9. Si Redis tarda más de 10 segundos, el backend deja de reintentar y continúa con memoria local.

6.3 Persistencia de datos

```
v24.18.0 B:\ghb\2026-10-TPI git:(develop) 3 • +37 -19 PR #1 (0.67s)
docker compose exec redis redis-cli SET prueba:persistencia "hola docker"
OK

v24.18.0 B:\ghb\2026-10-TPI git:(develop) 3 • +37 -19 PR #1 (0.93s)
docker compose stop redis

[+] stop 1/1
 ✔Container g10_m8_redis Stopped

v24.18.0 B:\ghb\2026-10-TPI git:(develop) 3 • +37 -19 PR #1 (0.865s)
docker compose start redis

[+] start 1/1
 ✔Container g10_m8_redis Started

v24.18.0 B:\ghb\2026-10-TPI git:(develop) 3 • +37 -19 PR #1 (0.57s)
docker compose exec redis redis-cli GET prueba:persistencia
"hola docker"
```

Evidencia 10. Redis: una clave escrita antes de detener el contenedor sigue disponible al volver a iniciarlo (volumen `redis_data` y `--appendonly yes`).



Evidencia 11. Alta de un comprobante desde el panel: código `REC-2026-XFT8569`.

```
v24.18.0 B:\ghb\2026-10-TPI git:(develop) 2 • +17 -7 PR #1 (13s)
docker compose exec postgres psql -U postgres -d m8_db

psql (16.15)
Type "help" for help.

m8_db=# SELECT "receiptCode", "tripId", amount, "createdAt" FROM "DocumentReceipt" WHERE "tripId" ='123';
   receiptCode    | tripId | amount  |        createdAt
------------------+--------+---------+------------------------
 REC-2026-XFT8569 | 123    | 3850.00 | 2026-11-10 04:38:47.178
(1 row)
```

Evidencia 12. El comprobante está guardado en PostgreSQL.

```
v24.18.0 B:\ghb\2026-10-TPI git:(develop) 2 • +17 -7 PR #1 (9.598s)
docker compose down

[+] down 9/9
 ✔ Container g10_m6_api_viajes           Removed
 ✔ Container g10_m8_frontend             Removed
 ✔ Container g10_m8_backend              Removed
 ✔ Container g10_m8_mailpit              Removed
 ✔ Container g10_m8_postgres             Removed
 ✔ Container g10_m8_redis                Removed
 ✔ Container g10_m8_rabbitmq             Removed
 ✔ Network 2026-10-tpi_m8_internal_net   Removed
 ✔ Network 2026-10-tpi_microservices_net Removed

v24.18.0 B:\ghb\2026-10-TPI git:(develop) 2 • +17 -7 PR #1 (30.736s)
docker compose up -d

[+] up 9/9
 ✔ Network 2026-10-tpi_microservices_net Created
 ✔ Network 2026-10-tpi_m8_internal_net   Created
 ✔ Container g10_m8_redis                Healthy
 ✔ Container g10_m8_rabbitmq             Healthy
 ✔ Container g10_m8_mailpit              Started
 ✔ Container g10_m8_postgres             Healthy
 ✔ Container g10_m8_backend              Healthy
 ✔ Container g10_m6_api_viajes           Started
 ✔ Container g10_m8_frontend             Started
```

Evidencia 13. `docker compose down` *elimina contenedores y redes;* `up -d` *los vuelve a crear. Los volúmenes no se borran.*



Evidencia 14. Después de recrear todo, el PDF del viaje 123 se descarga con los datos originales: la información sobrevivió en el volumen `postgres_data`.

6.4 Redes y aislamiento

```
v24.18.0 B:\ghb\2026-10-TPI git:(develop) 2 • +17 -7 PR #1 (0.195s)
docker network ls --filter name=2026-10-tpi
NETWORK ID     NAME                                DRIVER    SCOPE
1772d609fa91   2026-10-tpi_m8_internal_net     bridge    local
381c50dd2473   2026-10-tpi_microservices_net   bridge    local
```

Evidencia 15. Las dos redes creadas por Compose.

```
v24.18.0 B:\ghb\2026-10-TPI git:(develop) 2 • +17 -7 PR #1 (0.212s)
docker network inspect 2026-10-tpi_m8_internal_net --format "{{range .Containers}}{{.Name}} {{end}}"
g10_m8_backend g10_m8_postgres g10_m8_rabbitmq g10_m8_mailpit g10_m8_frontend g10_m8_redis

v24.18.0 B:\ghb\2026-10-TPI git:(develop) 2 • +17 -7 PR #1 (0.206s)
docker network inspect 2026-10-tpi_microservices_net --format "{{range .Containers}}{{.Name}} {{end}}"
g10_m8_backend g10_m8_rabbitmq g10_m6_api_viajes
```

Evidencia 16. Contenedores de cada red: la API de Viajes solo está en `microservices_net`.

```
v24.18.0 B:\ghb\2026-10-TPI git:(develop) 2 • +17 -7 PR #1 (0.687s)
docker compose exec api-viajes wget -qO- http://backend:3000/api/v1/health
{"estado":"ok","status":"ok","module":"M8 - Notificaciones, Documentos y Soporte","modulo":"M8 - Notificaciones, Documentos y Soporte","group":"Grupo 10","dependencias":{"postgres":"activo","redis":"activo"},"timestamp":"2026-11-10T04:47:20.791Z","uptime":321.457747646}
```

Evidencia 17. Desde `api-viajes` *se alcanza al backend por su nombre de servicio.*

```
v24.18.0 B:\ghb\2026-10-TPI git:(develop) 2 • +17 -7 PR #1 (2.875s)
docker compose exec api-viajes wget -qO- -T 3 http://postgres:5432
wget: bad address 'postgres:5432'
```

Evidencia 18. Desde `api-viajes` *no se resuelve* `postgres`*: no comparten red.*

```
v24.18.0 B:\ghb\2026-10-TPI git:(develop) 2 • +17 -7 PR #1 (3.258s)
docker compose exec api-viajes wget -qO- -T 3 http://redis:6379
wget: bad address 'redis:6379'
```

Evidencia 19. Tampoco se resuelve `redis`.

Aclaración

El aislamiento es por nombre. PostgreSQL y Redis publican sus puertos en el host (5432 y 6379) para poder usarlos en desarrollo, y un puerto publicado es alcanzable por IP. En un entorno productivo esos puertos no se publicarían.

6.5 Comunicación entre las dos APIs

La API de Viajes conoce al backend por una variable de entorno definida en Compose; el valor usa el nombre del servicio, que Docker resuelve dentro de la red compartida.

```yaml
api-viajes:
  build: ./apps/simulador-m6 # Construye la imagen con el Dockerfile de la carpeta del simulador
  container_name: g10_m6_api_viajes # Nombre del contenedor para identificarlo fácilmente
  restart: unless-stopped
  environment: # Sobreescribe solo estas del .env
    PUERTO: 3001
    URL_BACKEND_M8: http://backend:3000/api/v1
```

Evidencia 20. Definición de la segunda API en `docker-compose.yml`.

```
75          // Llamada HTTP contenedor-a-contenedor hacia M8 (backend)
76          const respuesta = await fetch(`${URL_BACKEND_M8}/simulator/trip-completed`, {
77            method: 'POST',
78            headers: { 'Content-Type': 'application/json' },
79            body: JSON.stringify(datos),
80          });
```

Evidencia 21. `apps/simulador-m6/servidor.js`*: llamada HTTP de contenedor a contenedor.*



Evidencia 22. «Finalizar viaje» desde la API de Viajes: el M8 responde con el código de comprobante y el destinatario notificado.



Evidencia 23. Mailpit recibe los correos generados por el M8.

6.6 Pruebas de integración con Testcontainers

El entorno se prepara una sola vez por corrida, en el `globalSetup` de Jest:

```
// apps/backend/src/integration-tests/support/postgres-test-environment.ts
container = await new PostgreSqlContainer('postgres:16-alpine')
  .withDatabase(TEMP_DB_NAME)
  .withUsername(TEMP_DB_USER)
  .withPassword(password)          // aleatoria por corrida
  .start();                        // sin puertos fijos: Docker asigna uno libre

const url = container.getConnectionUri();
assertTemporaryDatabaseUrl(url);   // guardián: nunca una base real
await applyMigrations(url);        // prisma migrate deploy
```

La prueba más representativa envía ocho altas simultáneas del mismo viaje y comprueba que exista un único comprobante:

```
// documents-receipts.integration.spec.ts
const responses = await Promise.all(
  Array.from({ length: 8 }, () =>
    ctx.http().post(RECEIPTS).send(validBody('trip-it-concurrent'))),
);
const ids = new Set(responses.map((res) => res.body.id));
expect(ids.size).toBe(1);
expect(await ctx.prisma.documentReceipt.count(
  { where: { tripId: 'trip-it-concurrent' } })).toBe(1);
```

Suite	Qué verifica
Comprobantes (7 casos)	Alta con persistencia real, repetición sin duplicados, 8 solicitudes simultáneas, viajes distintos, validación (400), descarga del PDF y viaje inexistente (404).
Tickets (5 casos)	Alta, alta inválida, lectura por identificador y por número, transiciones de estado y transiciones no permitidas.

Seguridad de la suite. La conexión sale únicamente del contenedor; no se carga ningún `.env`. Un guardián verifica usuario, base y un marcador de la corrida antes de cada TRUNCATE, y rechaza cualquier dirección que coincida con una base configurada en el entorno. SMTP, Redis y RabbitMQ se reemplazan por dobles de prueba: quedan fuera del alcance de esta suite.

```
npm run test:integration --workspace=@g10-m8/backend
```

Resultado de la ejecución. La suite se ejecutó el 11/11/2026 sobre el commit `c17a456` (rama `develop`), con Docker 29.9.2 y Node.js 24.20.0 en Windows 11. Las 12 pruebas resultaron aprobadas.

```
PASS src/integration-tests/documents-receipts.integration.spec.ts (6.559 s)
  Comprobantes PDF — API + Prisma + PostgreSQL temporal
    √ registra un comprobante: responde 201 y lo persiste realmente en PostgreSQL (343 ms)
    √ repetir el registro del mismo tripId devuelve el mismo comprobante y no crea duplicados (237 ms)
    √ solicitudes simultáneas del mismo tripId no duplican (advisory lock de PostgreSQL) (330 ms)
    √ viajes distintos generan comprobantes distintos con receiptCode único (236 ms)
    √ rechaza con 400 un cuerpo inválido y no persiste nada (178 ms)
    √ descarga el PDF de un comprobante existente sin crear ni modificar registros (262 ms)
    √ responde 404 al descargar un tripId inexistente y no crea registros (188 ms)

PASS src/integration-tests/support-tickets.integration.spec.ts
  Tickets de soporte — API + Prisma + PostgreSQL temporal (CRUD sobre endpoints existentes)
    √ CREATE: crea un ticket (201) en estado OPEN y lo persiste en PostgreSQL (233 ms)
    √ CREATE: rechaza con 400 un ticket inválido y no persiste nada (191 ms)
    √ READ: obtiene por UUID y por ticketNumber, lista con filtros y responde 404 si no existe (259 ms)
    √ UPDATE: recorre OPEN → IN_PROGRESS → RESOLVED → CLOSED y persiste cada estado (268 ms)
    √ UPDATE: rechaza con 400 una transición inválida y un ticket CLOSED; 404 si no existe (278 ms)

Test Suites: 2 passed, 2 total
Tests:       12 passed, 12 total
Snapshots:   0 total
Time:        9.288 s, estimated 12 s
Ran all test suites.
```

Evidencia 24. Salida de Jest en modo detallado: 2 suites y 12 pruebas aprobadas contra PostgreSQL real.

```
# Antes de la corrida
$ docker ps -a
(sin contenedores)

# Durante la corrida
$ docker ps
postgres:16-alpine   0.0.0.0:32771->5432/tcp   Up 1 second (health: starting)

# 20 segundos después de terminar
$ docker ps -a
(sin contenedores)
```

Evidencia 25. El contenedor efímero recibió el puerto 32771 *del host (no el 5432 fijo de Compose) y no quedaron contenedores al terminar: el propio Ryuk se eliminó a los pocos segundos.*

Qué demuestra esta corrida

Las migraciones existentes se aplican sobre PostgreSQL 16; el bloqueo `pg_advisory_xact_lock` evita duplicados ante 8 solicitudes simultáneas; y el entorno de pruebas no deja residuos ni usa puertos fijos.

6.7 Repetir la corrida: preparación del entorno y ciclo de vida

Para comprobar que la suite es reproducible, se la volvió a ejecutar el 11/11/2026 en otro entorno (Windows 11, Docker Desktop 29.9.2 sobre WSL2, Node.js 24.18.0), desde la rama `fix/m8-health-503-postgres` (commit b6238b9, ya integrado en `develop`). El resultado fue el mismo: 2 suites y **12 de 12 pruebas aprobadas** en 21 s (40 s en total, contando el arranque del contenedor). Durante la corrida se registró `docker ps` una vez por segundo:

```
# antes de la corrida
(sin contenedores)

# t+7 s · globalSetup crea la base y el contenedor auxiliar Ryuk
postgres:16-alpine           0.0.0.0:32771->5432/tcp   Up Less than a second (health: starting)
testcontainers/ryuk:0.14.0   0.0.0.0:32770->8080/tcp   Up 1 second

# t+10 s · la base pasa su healthcheck y corren las pruebas
postgres:16-alpine           0.0.0.0:32771->5432/tcp   Up 7 seconds (healthy)

# 5 s después de terminar · globalTeardown ya eliminó la base (docker ps -a)
testcontainers/ryuk:0.14.0                             Up 36 seconds

# segundos más tarde · Ryuk se retira solo (docker ps -a)
(sin contenedores)
```

Evidencia 26. Ciclo de vida observado en la segunda corrida: Docker volvió a asignar un puerto libre (32771), la base quedó healthy en unos 3 segundos, se eliminó al terminar y Ryuk, el contenedor que limpia recursos huérfanos, se retiró después.

La corrida falló tres veces antes de pasar. Ninguna falla fue de las pruebas: las tres fueron de preparación del entorno, y cada una apareció recién al resolver la anterior.

Falla observada	Causa	Cómo se resolvió
failed to connect to the docker API	Docker Desktop no estaba en ejecución. Testcontainers necesita un *daemon* de Docker: sin él, la suite no llega a arrancar.	Iniciar Docker Desktop y verificar con `docker info`.
TS2307: Cannot find module '@testcontainers/postgresql'	La dependencia está declarada en `package.json`, pero no estaba instalada en `node_modules`. La suite unitaria no la usa, por eso no se había notado.	Instalar las dependencias del monorepo (`npm install`).
the URL must start with the protocol prisma://	El cliente de Prisma generado no correspondía a una conexión directa a PostgreSQL. Se detectó al abrir la conexión real contra el contenedor.	Regenerar el cliente: `npm run prisma:generate`.

Qué enseña esta segunda corrida

Testcontainers hace reproducible **la base**, no **la máquina**: siguen haciendo falta Docker en ejecución, las dependencias instaladas y el cliente de Prisma generado. Son los pasos que figuran en la sección 8, y la razón por la que el siguiente paso es ejecutar la suite en el CI, donde esa preparación es siempre la misma. La tercera falla, además, solo se manifiesta con una conexión real: una prueba con un doble de Prisma no la habría mostrado.

7. Decisiones de diseño y hallazgos

Decisión o hallazgo	Fundamento
PostgreSQL caído → 503; Redis caído → 200 «degradado»	Sin base el M8 no puede cumplir su función, así que debe figurar como no disponible. Sin Redis puede seguir con memoria local, a costa de perder los QR pendientes si el backend se reinicia.
El backend no se reinicia aunque esté *unhealthy*	La política `restart` actúa cuando el proceso termina. El backend sigue vivo y se recupera solo cuando vuelve la base.
Reconexión a Redis limitada a 10 intentos	Evita reintentos infinitos. Como contrapartida, si Redis vuelve más tarde hay que reiniciar el backend para dejar la memoria local. Es una limitación conocida.
Dos redes en lugar de una	Separa lo privado del módulo de lo compartido entre módulos y hace visible la regla «cada servicio es dueño de sus datos».
Puertos de la base publicados en el host	Comodidad para desarrollo; debilita el aislamiento y no debe replicarse en producción.
Instalación de OpenSSL en la imagen	Prisma no encontraba su motor en `node:20-alpine` y el contenedor se reiniciaba en bucle. Se detectó recién al ejecutar el backend dentro de Docker.
`db push` **en el contenedor, migraciones en las pruebas**	El backend en Docker sincroniza el esquema con `prisma db push` al arrancar, cómodo para una demostración. Las pruebas aplican las migraciones versionadas, que es lo que correspondería en producción. El grupo verificó que ambos caminos producen el mismo esquema.
Sin endpoint DELETE para tickets	No existe en producción y no se agregó solo para la demostración; la limpieza entre pruebas se hace con TRUNCATE protegido.
Pruebas de integración fuera del CI	El pipeline actual valida la construcción de la imagen, pero no ejecuta la suite con Testcontainers; por ahora se corre en forma local. Sumarla al CI es el siguiente paso natural.

8. Cómo reproducir la demostración

Requisitos: Docker Desktop en ejecución y Node.js 20 o superior.

```
# Tema 14: sistema completo
docker compose up -d --build
docker compose ps
curl -i http://localhost:3000/api/v1/health

docker compose stop postgres       # el health pasa a 503 y el backend a unhealthy
docker compose start postgres
docker compose stop redis          # el health responde 200 con status "degradado"
docker compose start redis

docker network inspect 2026-10-tpi_microservices_net
docker compose exec api-viajes wget -qO- http://backend:3000/api/v1/health
docker compose down                # los volúmenes se conservan

# Tema 25: pruebas de integración
npm ci
npm run prisma:generate
npm run build --workspace=@g10-m8/contracts
npm run test:integration --workspace=@g10-m8/backend
```

Dirección	Qué muestra
http://localhost:3000/api/docs	Documentación Swagger del backend.
http://localhost:3001	API de Viajes (simulador M6) con los botones de la demostración.
http://localhost:5173	Panel web del M8.
http://localhost:8025	Correos capturados por Mailpit.
http://localhost:15672	Consola de RabbitMQ.

9. Conclusiones

- Docker Compose responde a la pregunta del Tema 14: un archivo versionado alcanza para que todo el grupo levante las dos APIs y sus cuatro dependencias con un comando, con redes, volúmenes y healthchecks declarados.
- Testcontainers responde a la del Tema 25: las pruebas crean y destruyen su propia base PostgreSQL, por lo que son repetibles y no dependen de ninguna base compartida.
- Los temas se complementan. Compose demuestra que el sistema *funciona en conjunto y resiste fallas*; Testcontainers demuestra que una funcionalidad *es correcta* contra la dependencia real. Usar la misma imagen y el mismo esquema en ambos mantiene la coherencia.
- Ninguno reemplaza al otro ni a las pruebas unitarias, y ninguno es una solución de producción: Compose no orquesta varios nodos y Testcontainers no prueba el sistema completo.
- El trabajo dejó a la vista decisiones que no aparecen en la teoría: qué dependencia es crítica, qué significa «disponible» y qué se sacrifica al publicar puertos.

10. Bibliografía

[1] Docker Inc. *Docker Compose documentation*. https://docs.docker.com/compose/

[2] Docker Inc. *Compose file reference*. https://docs.docker.com/reference/compose-file/

[3] Compose Specification. *The Compose Specification*. https://compose-spec.io/

[4] Docker Inc. *Networking in Compose* y *Volumes*. https://docs.docker.com/

[5] Testcontainers. *Testcontainers: getting started*. https://testcontainers.com/

[6] Testcontainers for Node.js. *Documentation* y *PostgreSQL module*. https://node.testcontainers.org/

[7] Fowler, M. (2018). *IntegrationTest*. https://martinfowler.com/bliki/IntegrationTest.html

[8] Vocke, H. (2018). *The Practical Test Pyramid*. https://martinfowler.com/articles/practical-test-pyramid.html

[9] Wiggins, A. (2011). *The Twelve-Factor App: X. Dev/prod parity*. https://12factor.net/dev-prod-parity

[10] Newman, S. (2021). *Building Microservices* (2.ª ed.). O'Reilly Media.

[11] Prisma. *Prisma Migrate*. https://www.prisma.io/docs/orm/prisma-migrate

[12] Grupo 10 (2026). Repositorio *FRRe-DS/2026-10-TPI*: `docker-compose.yml`, `docs/TESTCONTAINERS.md` y `apps/backend/src/integration-tests/`.

[13] Docker Inc. *What is a container?* https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/

[14] Docker Inc. *Docker overview*. https://docs.docker.com/get-started/docker-overview/

[15] Open Container Initiative. *About the OCI*. https://opencontainers.org/about/overview/

[16] Merkel, D. (2014). *Docker: lightweight Linux containers for consistent development and deployment*. Linux J., 239.